LECTURE 6 · A STORY IN THREE CHAPTERS

Testing AI-Generated Code, CI/CD Pipelines & Vercel Deployment

OrderMitra works on your laptop. Today we find out if it actually works — and teach a robot to check, every single time you push.

Instructor
Dept. of Computer Science and Engineering, IIT Kharagpur
mainack@cse.iitkgp.ac.in

Today's Story, in Three Chapters

Ch.1	Testing OrderMitra — Jest, React Testing Library & mocking AI
Ch.2	Can AI Write Our Tests? — AI-assisted generation & coverage audit
Ch.3	Shipping It — regression patterns, GitHub Actions CI & Vercel

By the final page, you'll be able to...

Reach 80%+ test coverage on OrderMitra, including deterministic tests for its non-deterministic AI features, and deploy through a CI pipeline that catches regressions automatically before they ship.

RECAP

The Story So Far

Five sessions in — here's the thread this one picks up.

- Session 1 — AI pair-programming discipline: specify, verify, override
- Session 2 — Next.js App Router, Server Components, strict TypeScript
- Session 3 — accessible components, verify what AI writes before it ships
- Session 4 — Prisma, OAuth, TanStack Query, review AI-drafted schemas
- Session 5 — OrderMitra: streams, searches orders, asks before it acts
- Today — prove all of that actually works, and teach a machine to keep checking

> *Every session has been "verify what AI writes." Today, verification itself gets automated.*

Quick Reminder: What Can OrderMitra Actually Do?

Before we talk about testing it, let's make sure we all remember what "it" is.

It Talks (Session 5, Ch.1-2)

- Streams answers word by word, using Server-Sent Events under the hood
- Shows a confidence badge — green/amber/red — next to answers it's unsure about

It Looks Things Up (Session 5, Ch.3)

- Calls a real tool, searchOrders, against your actual Prisma database
- Decides on its own when a question needs a database search

It Asks Before Acting (Session 5, Ch.4)

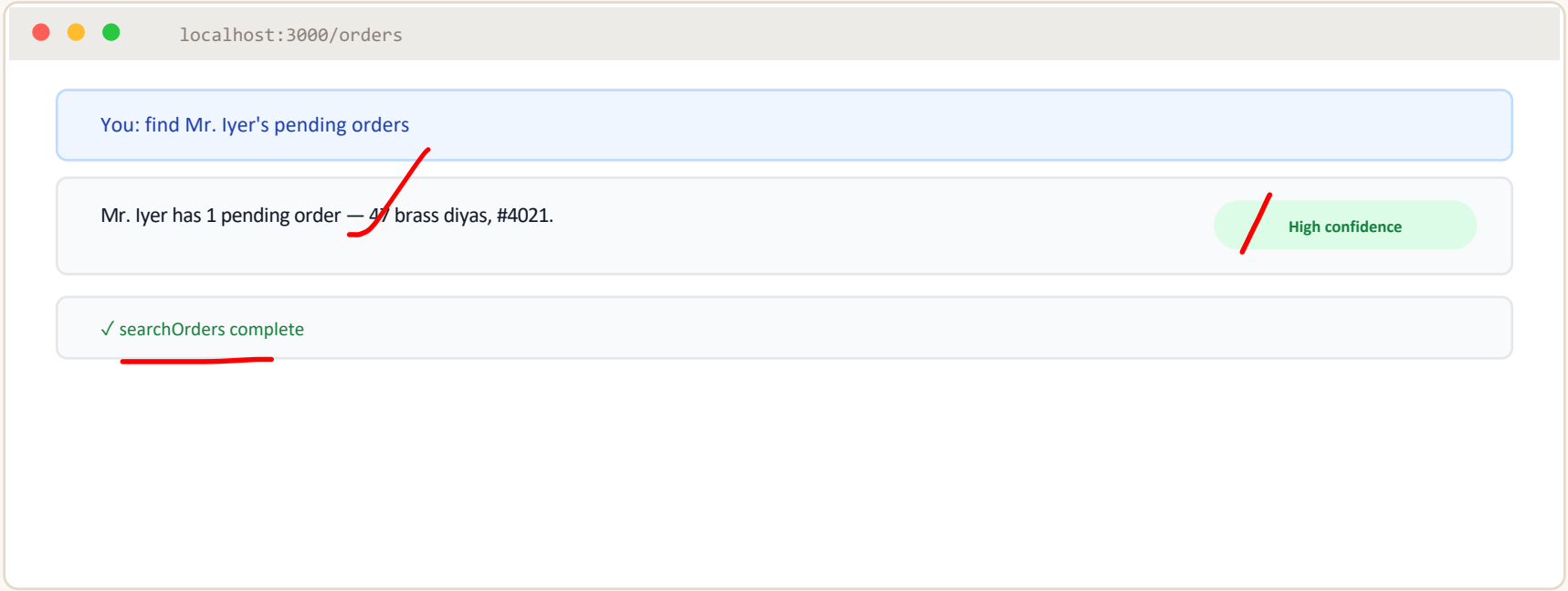
- cancelOrder is gated behind a confirmation card
- Nothing destructive runs until a human clicks Confirm

Three abilities, three different technologies underneath — which is exactly why testing it isn't one simple task, it's three different problems.

RECAP

What OrderMitra Looks Like Today

A quick visual reminder — this is the actual screen Session 5 left you with.



Three things happened on this one screen: a streamed reply, a confidence badge, and a real database search — all three need to be trustworthy, forever, not just today.

OrderMitra Needs to Ship

Now that we remember what it does — a very ordinary Monday morning conversation.

- OrderMitra works great — on your laptop, when you click around it yourself
- The team wants to actually launch it for real users
- Someone asks: "how do we know it still works after the next change?"
- Nobody has a good answer yet — that's this entire session

> *"It works on my machine" is not a deployment strategy — it's the joke every engineer has heard, right before the thing that broke it in someone else's hands.*

The Journey of One Code Change

Here's the whole session in one picture. We'll walk through every box, one at a time, over the next few slides.



Right now, only the first box exists for OrderMitra. By the end of today, every arrow is real.

Don't worry if some of these words are unfamiliar yet — that's the whole point of the next five slides.

Box 1 & 2: You Write Code, Tests Run

The first two boxes, in plain words, before we go any deeper into either.

"You Write Code"

- Exactly what you've done every session so far
- OrderMitra, running on your own laptop

"Tests Run"

- Small scripts that check your code actually works
- You run them yourself, on your laptop, before anyone else sees the change
- This is all of Chapter 1

Nothing new yet — just naming what you already do, plus one new habit: checking it automatically instead of by clicking around.

Box 3: git push

The one you already know from Sessions 2 through 5 — just named explicitly here.

In Plain Words

- The command that sends your committed code up to GitHub
- The moment your local change becomes visible to everyone else on the team
- Nothing automatic happens yet — pushing is just "sending"

This is the hinge point of the whole session — everything before this box is private to you; everything after it is automatic, and public to your team.

Box 4 & 5: GitHub Actions CI, Vercel Deploy

The two genuinely new boxes — a first pass now, the full story in Chapter 3.

"GitHub Actions CI"

- A robot that automatically re-runs your tests, on GitHub's own servers
- Runs the same checks from Box 2 — but nobody has to remember to trigger it

"Vercel Deploy"

- If the robot's checks pass, your change goes live automatically
- If they fail, nothing ships — real users never see the broken version

> *Boxes 1-3 are things you do. Boxes 4-5 are things that happen automatically, because you set them up once — that's the entire value of this chapter.*

CHAPTER 1

Testing OrderMitra: Jest, React Testing Library & Mocking AI

Zooming into Boxes 1 and 2 from the map — a test suite born from Facebook rewriting its chat feature in 2011 turns out to be exactly what a chat assistant needs, fifteen years later.

Try to Test OrderMitra by Hand, Completely

A genuine attempt — see how far "just click around it" actually gets you.

- Type a question. Check the reply looks right. Feels good so far.
- Try the confidence badge — high, medium, low. Three more clicks.
- Try searchOrders — with a name, without one, with a typo. A few more.
- Try cancelOrder — confirm it, reject it, click Confirm twice fast...
- ...and you've barely started. There are effectively infinite combinations left.

This isn't a hypothetical problem — it's the actual, physical limit of clicking around an app by hand. This is the real question Chapter 1 answers.

The Core Question This Chapter Answers

Say it plainly, because everything else in this chapter is just one long answer to it.

The Question

- You can never manually try every possible way a user could use OrderMitra
- So how do you actually know it works — beyond "it worked when I tried it"?

The Answer, in One Line

- Write down a fixed set of checks, once
- Let a computer run them, perfectly, in under a second
- Run that same set again after every single change, forever

> *You still can't test literally everything — nobody can. But you can test the things that matter, reliably, which is the entire difference this chapter is about.*

What Is a "Test," in Plain Words?

One definition, then three different concrete examples — so it's not just an abstract idea.

The Definition

- A small script that does something to your code, then checks the result
- If the check passes: green. If not: red, with exactly what went wrong

Three Examples of "Checks"

- "Does clicking Confirm actually call cancelOrder?"
- "Does the confidence badge show the right color for 'low'?"
- "Does an empty search return an empty list, not an error?"

Notice all three are things you'd already check by hand once — a test is just that same check, written down so it never has to be repeated manually again.

But How Does That Actually Work?

Not magic — a test genuinely simulates a tiny, fake version of the browser.

Step by Step

- 1. A fake, in-memory version of your component gets built — no real browser needed
- 2. The test script pokes at it: clicks a button, types text, whatever you tell it to
- 3. The test then looks at what's on screen now, and compares it to what you said should be there
- 4. Match = pass. No match = fail, with the exact difference shown

> *This is the whole trick: a test is a robot pretending to be a user, for one exact scenario, at superhuman speed.*

Who Actually Writes These?

A fair question before going further — this isn't a separate job from building the feature.

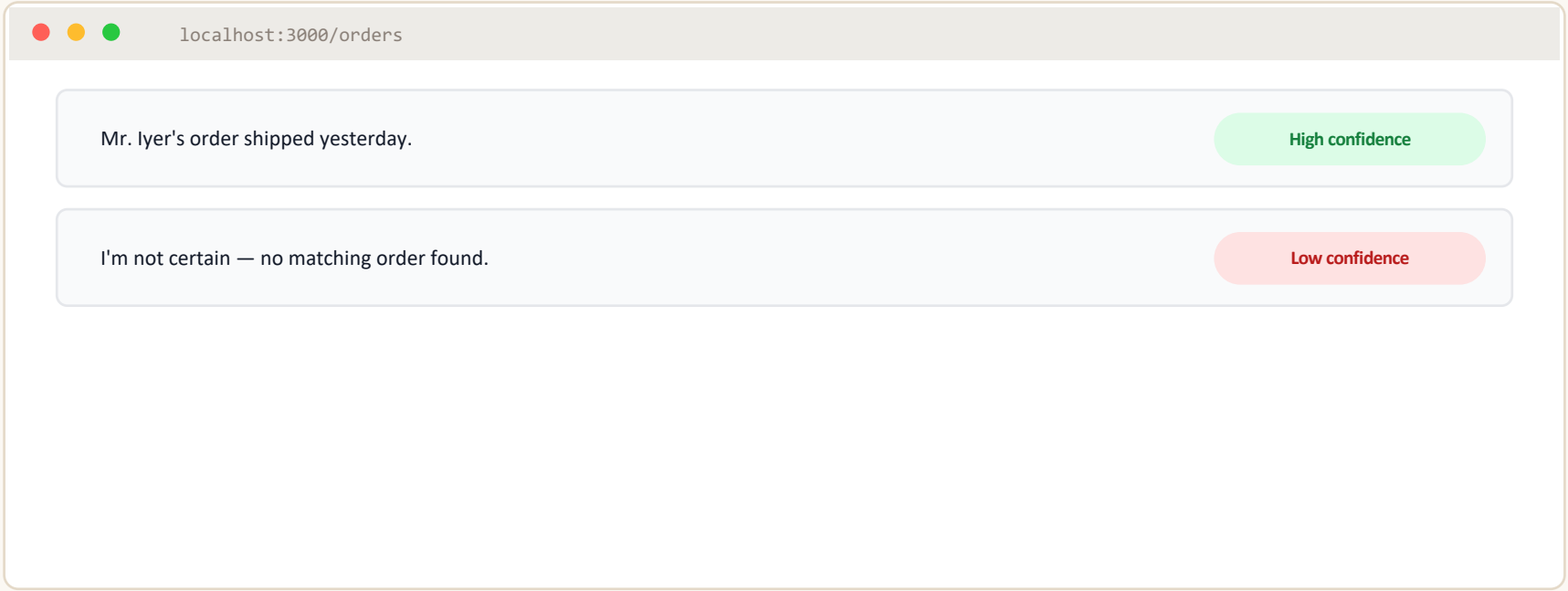
In Plain Words

- You do — the same person who wrote the feature
- Not a separate "QA team" writing tests after the fact, in this course's context
- Writing a test is part of finishing a feature, not an extra chore tacked on after

By the end of this chapter, "I built the feature" and "I tested the feature" will feel like the same sentence, not two separate ones.

Quick Reminder: What Is the Confidence Badge?

We're about to write a test for this exact Session 5 component — worth a 10-second visual refresh first.



A small component with one job: take a level ("high"/"medium"/"low") and show the right color and text. Small enough to be our very first test.

What Do We Actually Need to Write One of These?

Before naming any tool — what does "a test" from two slides ago actually require, mechanically?

Need #1

- Something to build the fake, in-memory component and run the checks
- This is a test runner

Need #2

- Something that knows how to "click" and "read text" on that fake component, the way a real user would
- This is a component-testing toolkit

This is true for any React project, not just OrderMitra — you always need both pieces. Jest and React Testing Library are just today's popular pairing for them.

How Jest Came to Be

- 2011 — Facebook rewrites its chat feature in JavaScript
- The added complexity demands a fast test-driven workflow
- 2014 — Jest is open-sourced, but maintained only part-time
- 2016 — Christoph Nakazawa builds a dedicated team, overhauls it
- Watch mode, snapshot testing, real developer experience — Jest becomes Jest

Jest exists because a chat feature got complicated — a strangely fitting origin for a tool about to test a chat assistant.

How Does Jest Actually Work?

Not magic — a fairly simple loop, run automatically every time you type npm test.

What Happens, Step by Step

- 1. Jest scans your project for any file ending in .test.ts or .test.tsx
- 2. Inside each one, it runs every test("...", () => {...}) block, one at a time
- 3. Each block's expect(...) calls either succeed or throw an error
- 4. Jest collects every result and prints one summary: how many passed, how many failed, and exactly why

That's the entire engine — find test files, run each function inside them, count the results. Everything else Jest offers (mocking, coverage, watch mode) builds on top of this one loop.

A FAMILIAR STORY

"It works when I click it" is exactly what everyone said about ~~OrderMitra~~ — right up until Mr. Iyer typed a diya order nobody had tried by hand.

Manual testing only ever covers what you remembered to try. Mr. Iyer's 47 diyas were never on anyone's list.

So Why Isn't Jest Enough on Its Own?

The bridge worth stating explicitly — Jest can run any JavaScript function. It has no idea what a React component even is.

What Jest Can Do Alone

- Run test("2+2 is 4", () => expect(2+2).toBe(4))
- Perfectly fine — but nothing there touches a component

What Jest Cannot Do Alone

- "Render <ConfidenceBadge /> and see what appears on screen"
- "Click this button, the way a real user would"
- Jest has no built-in concept of rendering or clicking at all

That gap — Jest runs code, but doesn't know how to build or interact with a component — is exactly what Need #2 from earlier was pointing at. React Testing Library is the tool built to fill it.

What Is React Testing Library, Specifically?

Jest is Need #1 — the runner. This is Need #2 — the toolkit that acts like a user.

In Plain Words

- A set of tools for rendering a component in a test, then querying it
- Built on one guiding idea: test it the way a real user would use it
- Find buttons by their visible text, not by internal implementation details

> *If a refactor changes how a component works internally but not what the user sees, RTL tests shouldn't break — that's the whole design philosophy.*

Where Do Test Files Actually Live?

A small practical question worth answering before writing a single test.

The Convention

- A test file sits next to the file it tests: confidence-badge.tsx + confidence-badge.test.tsx
- Jest automatically finds any file ending in .test.ts or .test.tsx
- No separate "tests folder" to keep in sync by hand

Keeping the test right beside the code it covers is a small habit with a big effect — far harder to forget updating a test you can see sitting right next to the change.

Every Test Has the Same Three Beats, Visually

"Arrange, Act, Assert" — a pattern so common it has a name. Here it is as an actual picture, not just words.

what happens inside one test

1. ARRANGE

Build the fake component on screen — like setting up a stage before the actors arrive.

2. ACT

Do ONE thing — click, type, submit — like a director calling "action."

3. ASSERT

Check what's now on screen matches what should be there.

Once you know to look for these three beats, you'll see them in almost every test you ever read — including the one on the very next slide.

Our First Test: The Confidence Badge

The three beats from the last slide, now in real code — Arrange, then Act, then Assert.

confidence-badge.test.tsx

```tsx
import { render, screen } from "@testing-library/react";
import { ConfidenceBadge } from "./confidence-badge";

test("shows high confidence in green", () => {
  render(<ConfidenceBadge level="high" />);   // Arrange
  const badge = screen.getByText("High confidence");  // Assert
  expect(badge).toBeInTheDocument();
});
```

What This Code Does

render() is Arrange — builds the badge in the fake browser • There's no Act this time — nothing to click, just check what rendered • screen.getByText() + expect() is Assert — confirms the right text showed up

AI Helps Here

Prompt: "Write a Jest + React Testing Library test confirming ConfidenceBadge renders 'High confidence' text when level is 'high'."

You Verify This

Confirm it's querying by visible text, not a CSS class — that's the RTL philosophy from two slides ago, in practice.

What Running That Test Actually Looks Like

terminal — npm test

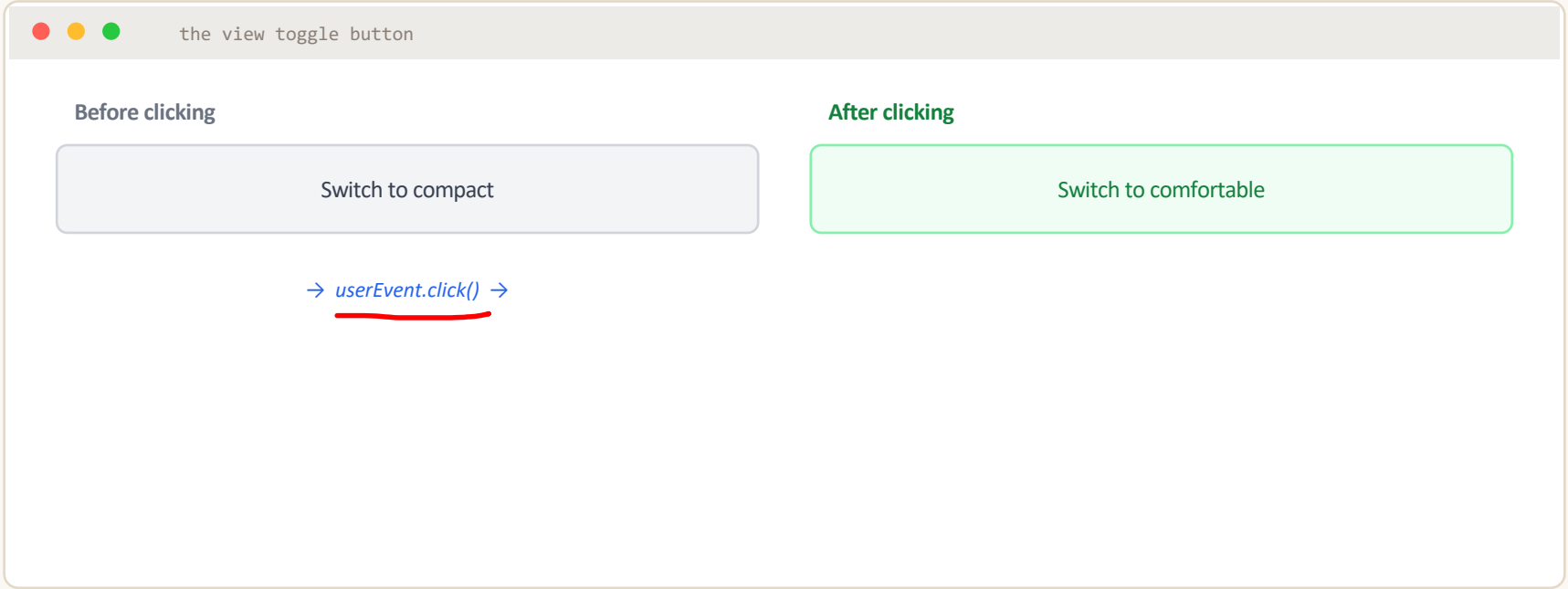
```
 PASS  confidence-badge.test.tsx
  ✓ shows high confidence in green (12 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Time:        0.841s
```

This green PASS is the entire payoff — a fact about your code, checked by a machine, in under a second.

Testing a Click, Visually — Before and After

"Testing a click" just means: check that the screen looks different after the click than it did before.



The test's job is exactly this picture: confirm the label really did flip, the same way your own eyes would confirm it.

The Same Click Test, in Code

The exact before/after picture from the last slide, written as Arrange, Act, Assert.

view-toggle.test.tsx

```tsx
import userEvent from "@testing-library/user-event";

test("click switches the label", async () => {
  render(<ViewToggle />);                                  // Arrange
  const button = screen.getByText("Switch to compact");
  await userEvent.click(button);                           // Act

  const flipped = screen.getByText("Switch to comfortable");
  expect(flipped).toBeInTheDocument();                     // Assert
});
```

What This Code Does

Arrange: render puts the "before" state on screen • Act: userEvent.click() is the arrow in the middle of the last slide's picture • Assert: checks for the "after" text — proof the click actually did something

AI Helps Here

Prompt: "Write a test that clicks the view toggle button and confirms its label switches from 'compact' to 'comfortable'."

You Verify This

userEvent, not fireEvent, for anything resembling a real user action — it simulates browser behavior more faithfully.

Back to the Question From the Start of This Chapter

Two tests written. A fair thing to wonder next.

- We wrote a test for the badge, and a test for a click
- OrderMitra has maybe a dozen more components and functions
- "Have we tested enough of it?" is now a real, answerable question
- We need a number, not a feeling

> *This is exactly the "how do you know you've tested enough" problem from the very start of this chapter — coverage is the tool built specifically to answer it.*

What Is "Coverage"?

The number that turns "I think we tested enough" into something you can actually check.

In Plain Words

- Coverage is the percentage of your code that tests actually run through
- 80% coverage means 80 lines out of 100 get executed by some test
- It's a floor to check, not a target to game — 100% coverage of trivial code proves nothing

A high coverage number tells you tests touched the code. It never tells you they checked the code was right — that part is still on you, always.

What a Coverage Report Actually Looks Like

terminal — npm test -- --coverage

```
File                  % Stmts   Uncovered Lines
confidence-badge.tsx  100%      —
view-toggle.tsx       100%      —
ordermitra-panel.tsx  0%        1-40
```

One glance tells you exactly what's covered and what isn't — no guessing, no "I think we tested that."

That Coverage Report Had a 0% in It

OrderMitra's actual chat panel — the one that talks to a real LLM — is completely untested. Let's fix that the obvious way.

- Same recipe as before: render it, ask a question, check the reply
- Let's try writing that test, exactly the way we wrote the last two
- Watch what happens when we actually run it

> *Sometimes the fastest way to understand why something is hard is to try the easy way first, and watch it fail.*

The Naive Attempt

Exactly the same pattern as the badge test — this looks like it should just work.

ordermitra-panel.test.tsx (the naive version)

```
test("replies to a question", async () => {
  render(<OrderMitraPanel />);
  fireEvent.click(screen.getByText("Ask about Mr. Iyer"));

  const reply = await screen.findByText("Mr. Iyer has 1 pending order.");
  expect(reply).toBeInTheDocument();
});
```

Run it once: passes. Run it again tomorrow: fails. The exact wording changed, even though nothing about OrderMitra actually broke.

What Does "Non-Deterministic" Mean?

The word for exactly what just happened — this test is fine, the assumption behind it isn't.

In Plain Words

- "Deterministic" = same input always gives the same output
- "Non-deterministic" = same input can give a different output each time
- 2 + 2 is deterministic. "Describe Mr. Iyer's orders" to an LLM is not
- Neither answer is wrong — they're just not identical, word for word

This is exactly why the naive test from the last slide is fragile — it assumed OrderMitra's real behavior, calling a real LLM, would be deterministic. It never was.

So What's the Fix?

We can't make the LLM deterministic — Anthropic controls that, not us. So we fix the test instead.

The Insight

- We were never actually trying to test the LLM's writing ability
- We were trying to test: does OUR code correctly display whatever comes back?
- That question doesn't need a real, unpredictable LLM at all

> *If the LLM's exact wording doesn't matter to what we're checking, replace it with something predictable, on purpose.*

What Does "Mocking" Mean?

The actual fix — and a very Bollywood way to think about it.

In Plain Words

- A mock is a fake stand-in that behaves predictably, used only in tests
- Think of it as a stunt double — looks and acts like the real thing, but you control exactly what it does
- Instead of calling the real Anthropic API, the test calls a fake version that always returns the same reply

> *We're not testing whether the LLM is smart — that's Anthropic's job. We're testing whether OrderMitra's code correctly handles whatever the LLM sends back.*

The Same Test, Fixed With a Mock

Compare this directly to the naive version a few slides ago — same shape, one key difference.

ordermitra-panel.test.tsx (fixed)

```
jest.mock("ai", () => ({
  useChat: () => ({
    messages: [
      { id: "1", role: "assistant", content: "1 order found." },
    ],
    status: "ready",
  }),
}));

test("renders the reply", () => {
  render(<OrderMitraPanel />);
  const reply = screen.getByText("1 order found.");
  expect(reply).toBeInTheDocument();
});
```

What This Code Does

jest.mock("ai") replaces the real SDK with our stunt double from the last slide • useChat now always returns the same fixed messages array, never a real LLM call • The component never touches the network — same result, every single run

AI Helps Here

Prompt: "Mock the 'ai' package's useChat hook to return a fixed assistant message, then test that OrderMitraPanel renders it."

You Verify This

Run it ten times in a row — confirm you get the exact same result every time. That repeatability is the entire point.

That Mocked Test Still Has a Hidden Assumption

Mocking fixed the wording problem. It did not fix a completely separate problem, still hiding in the same test.

- OrderMitra's reply doesn't appear the instant you click "Ask"
- It streams in over the next few hundred milliseconds — even a mocked one, once wired through useChat properly
- Our test checks for the text immediately after clicking, with no waiting at all
- Sometimes that's fast enough to pass. Sometimes it isn't. That's the new problem.

> *This has nothing to do with the LLM being unpredictable — it's about time. The reply is coming, just not synchronously, and our test needs to know how to wait.*

Two Separate Problems, Two Separate Fixes

Worth being explicit, since it's easy to blur these together.

Problem 1: Wording Changes

- Fixed by: mocking (the last three slides)
- Removes the LLM's unpredictability from the test entirely

Problem 2: Timing

- A completely different issue: WHEN the reply appears, not WHAT it says
- Needs a different fix — coming up next

Mocking a slow thing doesn't automatically make it instant — it just makes its content predictable. The waiting problem is still there.

The Fix for Timing: waitFor and findBy

RTL's built-in answer to "this will show up eventually, just not yet."

chat-panel.test.tsx

```
test("reply eventually appears", async () => {
  render(<OrderMitraPanel />);
  fireEvent.click(screen.getByText("Ask a question"));

  const reply = await screen.findByText("1 order found.");
  expect(reply).toBeInTheDocument();
});
```

What This Code Does

findByText is getByText's patient cousin — it retries until found or times out • await is required — this is genuinely asynchronous, not instant • No arbitrary setTimeout guessing — RTL polls efficiently on its own

AI Helps Here

Prompt: "Update the reply test to use findByText instead of getByText, since the message arrives asynchronously."

You Verify This

getByText throws immediately if not found; findByText waits. Mixing them up is the single most common async-testing mistake.

Two separate fixes for two separate problems: mock for the wording, findBy for the timing. Both, together, on the same test.

What Is "Snapshot Testing"?

One of Jest's own signature features — genuinely useful for some things, actively wrong for others.

In Plain Words

- Jest saves a "photo" of what a component rendered, the first time
- Every future test run compares against that saved photo
- Any difference — even a single character — fails the test and shows you exactly what changed

> *Think of it as a before-and-after photo comparison — perfect for catching accidental changes, useless for confirming the photo was ever correct in the first place.*

When Specifically to Use It

The direct answer to "when would I actually reach for this?"

Good Fit

- OrderMitra's page header — fixed markup, rarely changes
- A settings menu's layout — stable, same every render
- Catches: "oops, I accidentally deleted a wrapper div"

Bad Fit

- OrderMitra's chat reply — SUPPOSED to be different every time
- A snapshot of that content just fails constantly, for no real reason

Use snapshots for stable, rarely-changing markup — never for content that's meant to vary, like anything touching Chapter 1's non-determinism problem.

WHY IT'S CALLED A TRAP

A snapshot fails. The fastest fix is one command: jest --updateSnapshot. It also approves the failure as the new correct answer — including if the failure was a real bug.

A snapshot only protects you if a human actually reads the diff before approving it. Skip that, and it protects nothing at all.

Your Thumb Rule: What Pattern Fits What Component?

Not specific to OrderMitra — this is how to think about ANY component in ANY React project.

? Simple, deterministic component (a badge, a button)?	→ **Plain render + assert**
? Component with a user interaction (click, type)?	→ **Arrange, userEvent.click/type, Assert**
? Component that calls a real LLM or external API?	→ **Mock it first, then test your handling**
? Component whose content arrives asynchronously?	→ **findBy / waitFor, not getBy**
? Component with stable, rarely-changing markup?	→ **Snapshot testing**

If You're Testing By Hand: What's the Architecture?

One more generalization — the shape of a well-organized test file, for any component, in any project.

The Pattern

- One test file per component, sitting right beside it
- One test() block per behavior — not one giant test checking everything
- Arrange, Act, Assert, in that order, every single time
- Name each test as a sentence describing the behavior, not the implementation

Follow this shape and any new component becomes obvious to test — the pattern matters more than any single example.

One Problem Remains

A forward pointer, honestly stated.

- You now know how to write any of these tests, by hand
- OrderMitra has a dozen more components and functions still needing them
- Writing all of them, one at a time, by hand, is slow
- Chapter 2 asks the obvious next question: can AI help?

Everything in Chapter 2 builds directly on the patterns from this chapter — nothing gets thrown away, just sped up.

Aside: Where Else This Shows Up

Mocking non-deterministic dependencies is not an AI-specific trick.

You've Likely Seen This Already

- Mocking Date.now() so tests don't fail at midnight
- Mocking Math.random() for predictable dice-roll tests
- Mocking payment gateways in checkout flow tests
- Mocking any external API so tests don't need real network access

> *LLMs are just the newest thing on a long list of "this doesn't behave the same way twice" — the fix has been mocking all along.*

Chapter 1 Key Terms

Background material worth keeping.

Terms

- Test — a script that does something to your code, then checks the result
- Coverage — the percentage of code actually executed by tests
- Deterministic / non-deterministic — same input, same output vs. same input, varying output
- Mock — a predictable stand-in for something real, used only in tests
- Snapshot — a saved "photo" of rendered output, compared on every future run

END OF CHAPTER 1

Testing OrderMitra

- You can't manually test everything — a test is a check, written once, run forever
- Arrange, Act, Assert — the same three beats in every test you'll ever write
- Coverage tells you what's untested — not whether what's tested is actually correct
- Mock non-deterministic calls; use findBy/waitFor for anything asynchronous
- Snapshot only stable markup — never content that's supposed to vary

The takeaway: *Testing non-deterministic AI isn't a special new skill — it's the same mocking instinct engineers have used for random numbers and dates for years, aimed at a new kind of unpredictability.*

🔨 HANDS-ON STARTING

Test the Confidence Badge and Mock a Reply (https://shorturl.at/sworc)

~15 min

Apply this chapter's patterns yourself — a component test, then a mocked, async-aware one.

HANDS-ON COMPLETE

Two Real Tests, Both Green!

A component test and a mocked-hook test, both written by you. Back to the lecture — Chapter 2 asks whether AI can write these for us.

Today's Story, in Three Chapters

~~Testing OrderMitra — Jest, React Testing Library & mocking AI~~

Ch.2 Can AI Write Our Tests? — AI-assisted generation & coverage audit

Ch.3 Shipping It — regression patterns, GitHub Actions CI & Vercel

Up next

Can AI write these tests for us? Yes — and watching exactly what it forgets is the best lesson this session has to offer.

CHAPTER 2

Can AI Write Our Tests? Generation & the Coverage Audit

Chapter 1 taught you the patterns by hand. This chapter asks the obvious next question — and watches AI answer it almost perfectly.

Quick Recap: The Patterns From Chapter 1

Before asking AI to write anything — a reminder of exactly what "a good test" means, so we can judge what AI produces.

The Shape

- Arrange, Act, Assert — every single test
- One test file beside the component it covers
- One test() per behavior, named as a sentence

The Two Fixes

- Mock anything non-deterministic (a real LLM call)
- findBy/waitFor for anything that arrives asynchronously

Everything AI drafts in this chapter gets judged against exactly this list — it's the checklist, not a new set of rules.

The Obvious Next Question

You just spent 15 minutes writing two tests by hand. A reasonable person asks the obvious thing.

The Question

- "Can't the AI just write all of OrderMitra's tests for us?"
- We've used AI to draft schemas, components, tool definitions — why not tests?

The Honest Answer

- Yes — and it's genuinely good at the mechanical parts
- But "AI wrote it" was never the finish line in this course
- Session 4's schema chapter taught you why — same lesson, new artifact

This chapter is Session 4's "review the AI-drafted schema" lesson, aimed at test suites instead.

Step 1 — Ask AI to Write the Tests

The exact prompt, aimed at OrderMitra's chat panel — the same component Chapter 1 wrote two tests for by hand.

prompt to Claude Code

```
Write a full Jest + React Testing Library test suite for
OrderMitraPanel. Cover sending a message, showing the
typing indicator while streaming, and rendering the
assistant's reply.
```

AI Helps Here

This is a completely reasonable prompt — clear, scoped, and it's about to be answered well.

You Verify This

Notice what it doesn't mention: Stop, Retry, or anything about what happens when something goes wrong.

What the AI Actually Produced

Read it the way you would in a real review, before we point anything out.

ordermitra-panel.test.tsx (as generated)

```
test("sends a message and shows the reply", () => {
  render(<OrderMitraPanel />);
  fireEvent.change(screen.getByRole("textbox"), {
    target: { value: "find Mr. Iyer's orders" },
  });
  fireEvent.submit(screen.getByRole("form"));
  expect(screen.getByText(/typing/i)).toBeInTheDocument();
});

test("renders the assistant's reply", () => {
  // ...mocked response assertion
});
```

Two tests, both following Chapter 1's exact patterns — Arrange, Act, Assert, mocked correctly. Nothing here is wrong.

Step 2 — Run the Coverage Report

The number that reveals what the AI actually covered — and what it quietly skipped.

terminal — npm test -- --coverage

```
File                    % Stmts    Uncovered Lines

ordermitra-panel.tsx    76.4%      42-58


Lines 42-58: the abort() and reload() handlers —
never once called by any test.
```

76.4%, not 80% — and the missing lines are exactly Session 5's Stop and Retry buttons. The coverage report found what the prompt didn't ask for.

THE GAP

"Cover sending a message and showing the reply" is exactly what got tested. Nobody asked about stopping mid-stream, or retrying after a failure — so the AI, quite reasonably, never wrote for it.

This is the whole lesson of this chapter: the AI answered the prompt perfectly. The prompt just wasn't the whole spec.

Why AI Systematically Misses This

Not a bug in the model — a predictable pattern worth understanding.

What It's Good At

- The "happy path" — the obvious, intended use case
- Whatever's explicitly named in the prompt
- Boilerplate test structure and setup, following Chapter 1's shape correctly

What It Systematically Skips

- Edge cases nobody thought to mention
- Error states, cancellations, retries
- Anything that requires imagining a user doing something unexpected

This is the same pattern from Session 4's schema chapter and Session 5's tool definitions — AI does exactly what's asked, precisely, and the gaps live in what was never asked.

Step 3 — Write the Missing Test Yourself

The fix for the actual gap the coverage report found — this is the one a human had to notice.

ordermitra-panel.test.tsx (the fix)

```
test("stop button halts a streaming reply", () => {
  render(<OrderMitraPanel />);
  fireEvent.click(screen.getByText("Ask a question"));
  fireEvent.click(screen.getByText("Stop generating"));
  expect(mockStop).toHaveBeenCalled();
});
```

What This Code Does

Follows Chapter 1's exact shape — Arrange, Act, Assert • Directly targets the two uncovered lines from the coverage report • Simulates the exact user action (clicking Stop) that the AI's tests never tried

AI Helps Here

Prompt: "Add tests for the Stop and Retry buttons in OrderMitraPanel, confirming they call stop() and reload() from useChat."

You Verify This

This prompt exists BECAUSE you read the coverage report — the audit came first, the fix came second.

How to Prompt AI to Write Tests More Efficiently

Directly useful, generalizable advice — not specific to this one gap.

Weak Prompts

- "Write tests for this component"
- Vague scope — AI guesses what matters

Strong Prompts

- Name the happy path AND explicitly ask for error/edge states
- Reference Chapter 1's patterns directly: "mock the LLM call, use findBy for async content"
- Ask it to list what it did NOT cover, at the end

A stronger prompt narrows the gap — it has never once eliminated the need to run coverage and actually look. Both habits stay, always.

How to Actually Improve a Low Coverage Number

A concrete, repeatable process — not "write more tests" as vague advice.

The Process

- 1. Run npm test -- --coverage, read the uncovered line numbers
- 2. Open those exact lines — what do they actually do?
- 3. Write one focused test per uncovered behavior, not one giant test
- 4. Re-run coverage — confirm the number actually moved

> *"Improve coverage" sounds vague until you realize it's just: read the red lines, understand them, write one test each. Nothing more mysterious than that.*

What Is a "Coverage Threshold"?

The 80% target from this session's learning outcome, made concrete.

In Plain Words

- A minimum coverage percentage a project agrees to maintain
- Below the threshold, CI fails the build — even if every test passes
- 80% is a common, reasonable default: thorough, without chasing the last unreachable lines

A threshold turns "we should really improve our tests sometime" into a number a machine enforces automatically — no relying on good intentions. Chapter 3 shows exactly where this plugs in.

A CAUTIONARY TALE

A team once hit 100% coverage by writing one test per function that called it and asserted nothing at all. Every line ran. Nothing was actually checked. The bugs shipped anyway.

Coverage answers "did tests touch this code?" It has never once answered "is this code correct?" — that's still on you.

The Same Loop, a Different Component

Worth seeing this pattern land twice before it's fully trusted — this time on Session 5's searchOrders tool.

- Prompt: "Write tests for the searchOrders tool function"
- AI drafts a test: search by customer name, confirm the right order comes back
- Coverage report: 68% — the "no orders found" branch is never exercised
- The gap: nobody asked what happens when a search finds nothing at all

Different component, same shape of gap — AI tested what a search returns, not what a search fails to return.

Closing That Gap

search-orders.test.ts

```
test("returns an empty array when nothing matches", async () => {
  const results = await searchOrders.execute({
    customerName: "Nobody With This Name",
  });
  expect(results).toEqual([]);
});
```

What This Code Does

Arrange: call execute() with a name guaranteed not to match • Act: the search runs against the (mocked) database • Assert: an empty array, not undefined or an error — the shape OrderMitra's UI expects

AI Helps Here

Prompt: "Add a test confirming searchOrders returns an empty array, not an error, when no orders match."

You Verify This

Check what OrderMitra's chat actually does with an empty array — does it say "no orders found," or does it silently say nothing?

Two components, two audits, the exact same missing category both times: what happens when the answer is "nothing"?

A Battery of Prompts: What AI Handles Well

Not one example — an actual reference set. Use these shapes for anything with a single, obvious correct behavior.

Rendering & Props

- "Write a test confirming [Component] renders [text] when [prop] is [value]."
- "Test that [Component] renders nothing when [prop] is undefined."

User Interaction

- "Write a test that clicks [button] and confirms [expected UI change]."
- "Test typing into [input] updates its displayed value."

Pure Functions

- "Write tests for [function] covering typical inputs and their expected outputs."
- "Test that [function] throws when given [invalid input]."

Notice the pattern: every one of these has ONE unambiguous right answer. That's exactly the category AI drafts reliably.

A Battery of Prompts: Where You Must Be Explicit

AI can handle these too — but only if you name them. Left unnamed, Chapter 2's whole story shows they get skipped.

Edge Cases

- "...and also test the empty-input case, and what happens when the list is empty."
- "...including what happens if the API call fails or times out."

Async & Streaming

- "...using findBy/waitFor, since the content arrives asynchronously."
- "Test the loading state BEFORE the data arrives, not just after."

State Transitions

- "Test clicking Stop while streaming, and Retry after an error."
- "Test what happens if the user clicks Confirm twice, quickly."

This list is basically Chapter 1's patterns, restated as things to explicitly ask for — the more of these you name, the smaller the gap Chapter 2's audit has to catch.

What NOT to Hand Entirely to AI

Not "AI can't help here" — more precisely, these need a human decision AI cannot make for you.

Deciding What's Correct

- AI can write a test for behavior you describe — it cannot invent the correct business rule
- "Should a cancelled order still show in search results?" is a product decision, not a testing one

Security-Sensitive Logic

- Auth checks, permission gates, anything protecting real user data
- Write these tests yourself, or review an AI draft line by line, every time — no exceptions

Code You Don't Understand Yourself

- If you can't explain what a function should do, you can't judge whether AI's test is actually testing the right thing
- Understand it first, then delegate the typing

The dividing line isn't "simple vs complex" — it's "is there one unambiguous right answer" vs "does this require judgment only a human has."

Your Thumb Rule: Delegate or Write It Yourself?

? Rendering, props, simple clicks — one obvious right answer?	→ **Delegate, AI is reliable here**
? Edge cases, async, state transitions?	→ **Delegate, but name them explicitly in the prompt**
? Security, permissions, real user data?	→ **Write it yourself, or review every line**
? You can't explain the correct behavior yourself?	→ **Understand it first — don't delegate confusion**

Your Thumb Rule: Auditing an AI-Drafted Test Suite

? Coverage report shows uncovered lines?	→ Read what they actually do before writing a test
? Only happy-path tests exist?	→ Ask: what's the error state? The cancel? The retry?
? Tests only check what the prompt named?	→ Assume anything unnamed is untested
? Coverage is 100% but bugs still slip through?	→ Coverage counts lines run, not correctness — check assertions, not just %

Aside: Where Else This Shows Up

"AI covers the obvious case, misses the edge case" is not specific to testing.

The Same Pattern, Elsewhere

- AI-generated schemas: happy-path types, missing indexes (Session 4)
- AI-generated tool definitions: missing error handling (Session 5)
- AI code review bots: flag style issues, miss logic bugs
- AI-written documentation: covers the feature, skips the caveat

Once you've seen this pattern once, you start checking for it everywhere an AI hands you a finished-looking artifact.

Chapter 2 Key Terms

Background material worth keeping.

Terms

- Coverage threshold — the minimum % a project requires before CI passes
- Coverage gap — code with no test touching it at all
- Happy path — the intended, obvious way a feature gets used
- Edge case — an unusual, easy-to-forget situation a feature must still handle

END OF CHAPTER 2

Can AI Write Our Tests?

- Yes — AI drafts the mechanical, obvious-path tests quickly and well, following Chapter 1's patterns
- Coverage reports reveal exactly what got skipped, line by line
- AI systematically misses edge cases nobody explicitly asked for
- Draft, audit the report, write the missing test — the loop that closes the gap
- A stronger, more specific prompt narrows the gap; it never replaces the audit

The takeaway: *An AI-drafted test suite is a fast first draft, exactly like an AI-drafted schema or component — never the finished, trusted artifact on its own.*

🔨 HANDS-ON STARTING

Draft, Audit, and Close a Coverage Gap

~15 min

Repeat this chapter's exact loop on your own machine — draft, discover the gap, fix it.



HANDS-ON COMPLETE

You Found the Gap Yourself!

Draft, audit, fix — the exact loop from the chapter, now a habit instead of a story. Back to the lecture for what happens after tests pass locally.

Today's Story, in Three Chapters



~~Testing OrderMitra — Jest, React Testing Library & mocking AI~~



~~Can AI Write Our Tests? — AI-assisted generation & coverage audit~~

Ch.3 Shipping It — regression patterns, GitHub Actions CI & Vercel

Up next

Tests are green on your laptop. Chapter 3 makes sure they stay green — automatically, on every single push, before anything reaches a real user.

CHAPTER 3

Shipping It: Regression Patterns, GitHub Actions CI & Vercel

Your tests are green today. Chapter 3 is about the scarier question: will they still be green next month, when you're not even looking?

A Very Ordinary Friday

A small, completely believable story — no villains, just a normal week.

- A teammate finishes a small OrderMitra fix late Friday afternoon
- They're confident, in a hurry to leave — they don't run the test suite locally
- They push. Nobody stops them — nothing currently could
- Monday morning: cancelOrder has been silently broken all weekend

> *Nobody did anything malicious. The tests from Chapters 1 and 2 existed the whole time — they just never got a chance to run.*

What Would Have Actually Stopped This?

Not "tell people to be more careful" — that's not a system, that's a hope.

What We Actually Need

- Tests that run automatically, on EVERY push — not "whenever someone remembers"
- Something that physically blocks a broken change from reaching real users
- No human willpower required, ever

That description — automatic, on every push, blocking — is about to get a name. Everything else in this chapter is just filling in the details of that one idea.

A Story: The Bug Nobody Wrote

Before naming it — here's exactly what a "regression" actually feels like from the inside.

- Last month: cancelOrder worked perfectly, tested and confirmed
- This month: it silently stops working
- Nobody touched cancelOrder's code. git blame shows nothing recent, for anyone
- So... what actually broke it?

> *This is the strange, specific fear of a regression: your own code is innocent, and it broke anyway.*

What Is a "Regression"?

The name for the story you just read.

In Plain Words

- Something that used to work, and quietly stopped, without anyone deliberately breaking it
- "Regression" because the software regressed — went backward — from working to broken
- The scary part: it can happen with zero changes to the responsible code at all

Ordinary bugs point you at a diff. A regression sometimes points you at nothing — which is exactly why it needs its own chapter.

How, Then? Three Ways This Actually Happens

Three real categories, each with its own tiny story.

Prompt Drift

- A teammate tweaks OrderMitra's system prompt for one unrelated bug
- It subtly changes how EVERY other feature answers too

Model Version Change

- Anthropic quietly updates the model behind claude-sonnet-4-6
- The exact same prompt now behaves slightly differently — nothing in your code moved

Tool Call Failure

- A rate limit or timeout silently turns a tool call into a failure
- OrderMitra keeps running, just wrong, with no error anyone notices

Three different causes, one identical symptom: working code, stopped working, and git blame shows nothing.

What Is "Prompt Drift," Specifically?

The first of the three, worth a slightly closer look since it's the one most in your own control.

In Plain Words

- The same prompt, over time, starts producing subtly different results
- Sometimes it's the provider's model changing underneath you
- Sometimes it's your OWN prompt slowly growing extra instructions, added one bug fix at a time, that nobody reviewed together

Prompt drift is invisible until a test built specifically to catch it starts failing — which is the entire reason to build one.

Your Thumb Rule: What to Regression-Test for AI Features

Always Test

- Tool calls fire with correctly-shaped arguments
- Confirmation gates still block before destructive tools run
- A malformed or empty model response doesn't crash the UI

Consider Testing

- A snapshot of expected response "shape" (fields present, not exact wording)
- Response time stays under an acceptable threshold

You can't test that an LLM's wording never changes — so don't. Test the shape and behavior around it, which should stay stable.

Back to "What Would Have Stopped Friday"

We already answered this a few slides ago — now let's actually build it.

What We Said We Needed

- Tests that run automatically, on EVERY push
- Something that physically blocks a broken change from shipping
- No human willpower required

That description has a name. It's called Continuous Integration — "continuous" because it never skips a push, "integration" because it checks your change fits with everything else before it merges.

What Is "CI"?

Now that the need is established — the actual, precise definition.

In Plain Words

- Every time code is pushed, a machine automatically runs your tests
- Not "whenever someone remembers to" — every single time, no exceptions
- If tests fail, the machine says so, before broken code reaches anyone else

CI doesn't write better code — it makes sure the tests you already wrote in Chapters 1 and 2 actually get run, reliably, forever.

Where Does GitHub Actions Come From?

A fair question — why this particular tool, and not something else?

- October 2018 — GitHub announces Actions at GitHub Universe, limited beta
- November 2019 — rolled out generally, to every public repository
- The pitch: automate any workflow, directly inside your repo — no separate service to wire up
- Today — the default way most teams run tests before merging code

> *Before this, running tests "automatically" meant paying for and configuring a completely separate service. GitHub Actions folded it into the same place your code already lives — which is why it won.*

What Is YAML, Actually?

Every GitHub Actions workflow is written in this — worth more than a passing mention.

In Plain Words

- A plain-text way to write structured configuration
- Indentation shows nesting — no braces {}, no semicolons
- Two spaces in, means "this belongs inside the thing above it"

The Same Idea, in JSON

- { "name": "Test", "on": ["push"] }
- YAML says the same thing with indentation instead of punctuation:
- name: Test
- on: [push]

If the indentation is wrong, YAML silently means something different — not an error, just a different structure. Get in the habit of checking it carefully.

A YAML File, Built Up in Three Small Steps

Rather than reading one big file cold — watch it grow, one idea at a time.

Step 1: Just a name

- name: Test
- (That's the whole file. It does nothing yet.)

Step 2: When to run

- name: Test
- on: [push, pull_request]
- (Still does nothing — just says WHEN.)

Step 3: What to actually do

- Add jobs: underneath — the next slide's full file
- This is where the real work gets defined

Every real workflow file is just this: a name, a trigger, and a list of jobs — nothing more exotic than that, however long it looks.

npm ci vs npm install: Why Both Exist

A small but real distinction, worth understanding before the workflow file uses one of them.

npm install (local)

- Can update package-lock.json if versions have drifted
- Fine on your own laptop, where you're the only one affected

npm ci (CI machines)

- Installs the EXACT versions already locked — refuses to update anything
- If the lockfile and package.json disagree, it fails loudly instead of guessing

A CI machine should behave identically every single run, with zero surprises — npm ci exists specifically to guarantee that.

Exact Steps: Get OrderMitra Onto GitHub

Before any workflow file can run, GitHub needs to actually have your code. Five concrete steps, if you haven't done this yet.

On github.com

- 1. Click the + icon (top right) → "New repository"
- 2. Name it, e.g. ordermitra — choose Private or Public
- 3. Leave "Add a README" unchecked if you already have local code
- 4. Click "Create repository" — GitHub shows you a remote URL

This URL is what your local project will push to next — copy it, you'll need it on the very next slide.

Exact Steps: Push Your Local Code

Step 5 — connecting your existing OrderMitra project to the repo you just created.

```
terminal, inside your project folder

git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/you/ordermitra.git
git push -u origin main
```

What This Code Does

git init starts version control in this folder, if not already done • add + commit stages and saves everything as one snapshot • remote add points your local repo at the GitHub URL from the last slide • push -u origin main uploads it, and remembers this connection for next time

AI Helps Here

Prompt: "Give me the exact git commands to initialize this project and push it to a new GitHub repo at [your URL]."

You Verify This

Refresh the GitHub page — confirm your actual files appear, not just an empty repo.

Exact Steps: Add the Workflow File

One more step before the workflow on the next slide can actually run.

In Your Project

- 1. Create the folder path exactly: .github/workflows/
- 2. Inside it, create a file named test.yml (any name works, but this is conventional)
- 3. Paste in the workflow from the next slide
- 4. Commit and push it, the same way as the last slide

GitHub only looks inside .github/workflows/ for Actions — a file anywhere else is simply ignored, silently.

The Complete Workflow for OrderMitra

Step 3 from two slides ago, written out in full — you already know what every line means.

.github/workflows/test.yml

```yaml
name: Test
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm test -- --coverage
```

What This Code Does

name + on: exactly Steps 1-2 from two slides ago • runs-on gives it a fresh Ubuntu machine, every run • checkout + setup-node prepare the environment from scratch • npm ci (not install!) then npm test — the exact commands from your own laptop

AI Helps Here

Prompt: "Write a GitHub Actions workflow that runs on every push and pull request: checkout, set up Node 20, install dependencies with npm ci, and run tests with coverage."

You Verify This

Confirm node-version matches what you actually develop with locally — a mismatch here causes "works on my machine" failures inside CI itself.

What About the API Key? GitHub Secrets

Since Chapter 1's tests mock the LLM, CI doesn't need a real key today — but any real project eventually does, so it's worth knowing now.

The Wrong Way

- Hardcoding ANTHROPIC_API_KEY directly in the workflow file
- That file is committed to the repo — the key leaks the moment it's public

The Right Way

- Store the key in GitHub repo Settings → Secrets
- Reference it as ${{ secrets.ANTHROPIC_API_KEY }} in the workflow
- GitHub injects it at run time — it's never visible in the file itself

> *This is the exact same instinct as .env.local from Session 4 — the secret lives outside version control, always.*

Exact Steps: Add a Secret

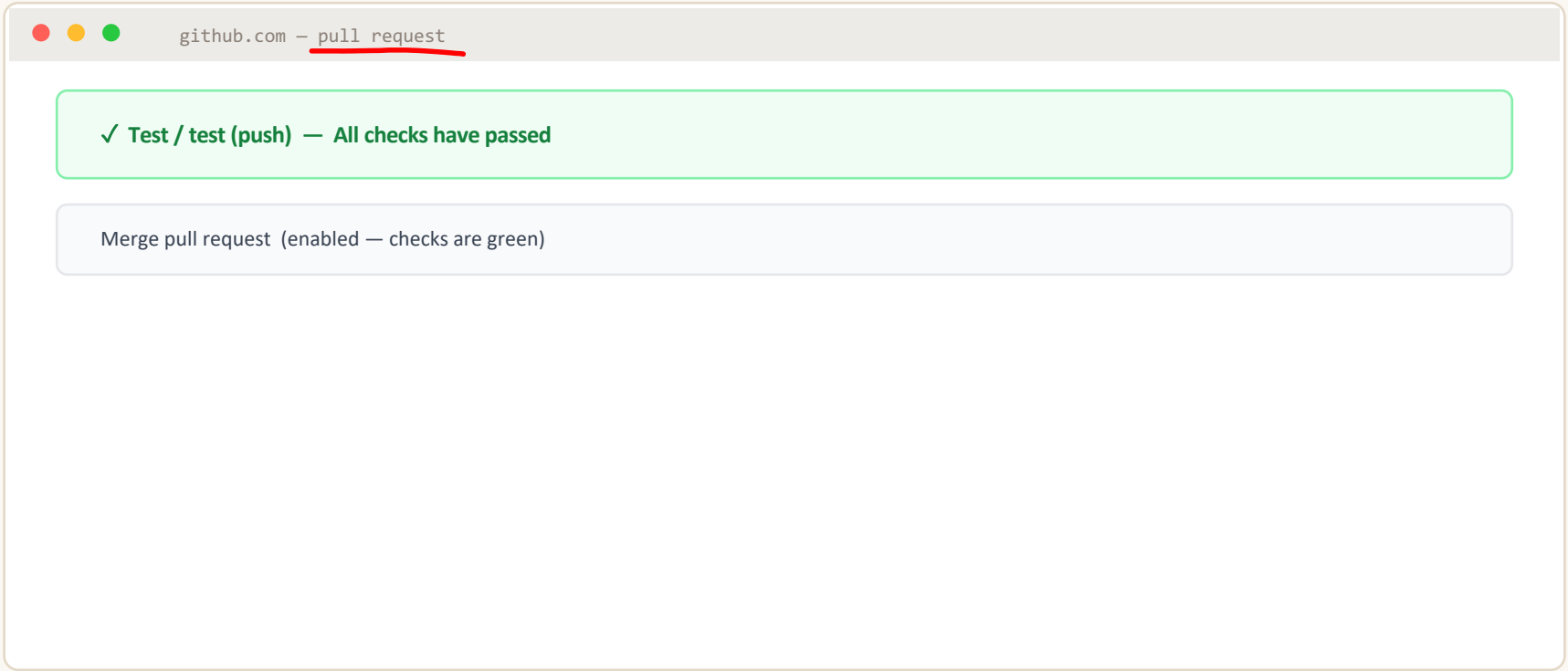
Four clicks, if your project ever does need a real key in CI.

On Your Repo Page

- 1. Settings tab → "Secrets and variables" (left sidebar) → Actions
- 2. Click "New repository secret"
- 3. Name: ANTHROPIC_API_KEY. Value: paste the real key
- 4. Click "Add secret" — it's now usable, never visible again, even to you

Once saved, GitHub only ever shows you the name again, never the value — the same one-way behavior as a password field.

What This Looks Like on GitHub



This green checkmark is what a teammate sees before merging — not "I ran it locally," but "a machine ran it, just now, and confirmed."

Making the Green Check Mandatory: Branch Protection

Without this one setting, that green check is just a suggestion — anyone can merge past a red one anyway.

In Plain Words

- A GitHub repo setting: Settings → Branches → Branch protection rules
- "Require status checks to pass before merging" — points at your CI workflow by name
- Once enabled, the Merge button is greyed out until CI is actually green — no exceptions

This is the one setting that turns "we have a CI pipeline" into "nothing merges without it" — the difference between a suggestion and a real gate.

Exact Steps: Turn It On

Five clicks — the setting from the last slide, done for real.

On Your Repo Page

- 1. Settings tab → Branches (left sidebar)
- 2. "Add branch protection rule" → Branch name pattern: main
- 3. Check "Require status checks to pass before merging"
- 4. Search for and select your Test workflow from the list
- 5. Save changes

From this point on, even the repo owner cannot merge a red pull request into main — the gate applies to everyone, no exceptions.

We've Covered the "CI" — What's the "CD"?

A term we've been circling without ever naming directly — worth stopping and defining properly.

CI — Continuous Integration

- Automatically testing every change (everything so far this chapter)

CD — Continuous Deployment

- Automatically shipping every change that passes CI
- No manual "click deploy" step — a green check IS the deploy trigger

CI decides whether a change is safe. CD is what happens the instant it's decided: it just... ships, on its own.

What Is Vercel Deployment?

The CD half, made concrete — turning a green check into a live URL, automatically.

In Plain Words

- Vercel connects directly to your GitHub repo
- Every push to main can trigger a fresh production deployment
- Every pull request can get its own preview URL, automatically
- No manual "upload the build" step, ever

Recall Session 4: Vercel is the same company behind the Next.js team and the AI SDK — deployment is their other core product, built for exactly this framework.

Exact Steps: Connect Vercel to Your Repo

This only needs to happen once per project — five steps, using the GitHub repo from earlier in this chapter.

First-Time Setup

- 1. Go to vercel.com → Sign up / log in with your GitHub account
- 2. Click "Add New..." → "Project"
- 3. Find and "Import" your ordermitra repo from the list
- 4. Vercel auto-detects it's a Next.js app — build settings need no changes
- 5. Click "Deploy"

That's it for the very first deploy — Vercel builds the project exactly the way npm run build would locally, then puts it on a live URL.

Exact Steps: Add Environment Variables

Your real ANTHROPIC_API_KEY and DATABASE_URL need to exist here too — Vercel doesn't read your local .env.local file.

In the Vercel Dashboard

- 1. Open your project → Settings tab → Environment Variables
- 2. Add each one: Key = ANTHROPIC_API_KEY, Value = your real key
- 3. Repeat for DATABASE_URL, and anything else from your .env.local
- 4. Choose which environments each applies to: Production, Preview, or both
- 5. Redeploy — new environment variables only take effect on the next build

This is the third place your secrets now live — .env.local for you, GitHub Secrets for CI, and this for the live app — each one deliberately separate.

Exact Steps: What Happens on Every Future Push

The one-time setup is done. From here, this is fully automatic, forever.

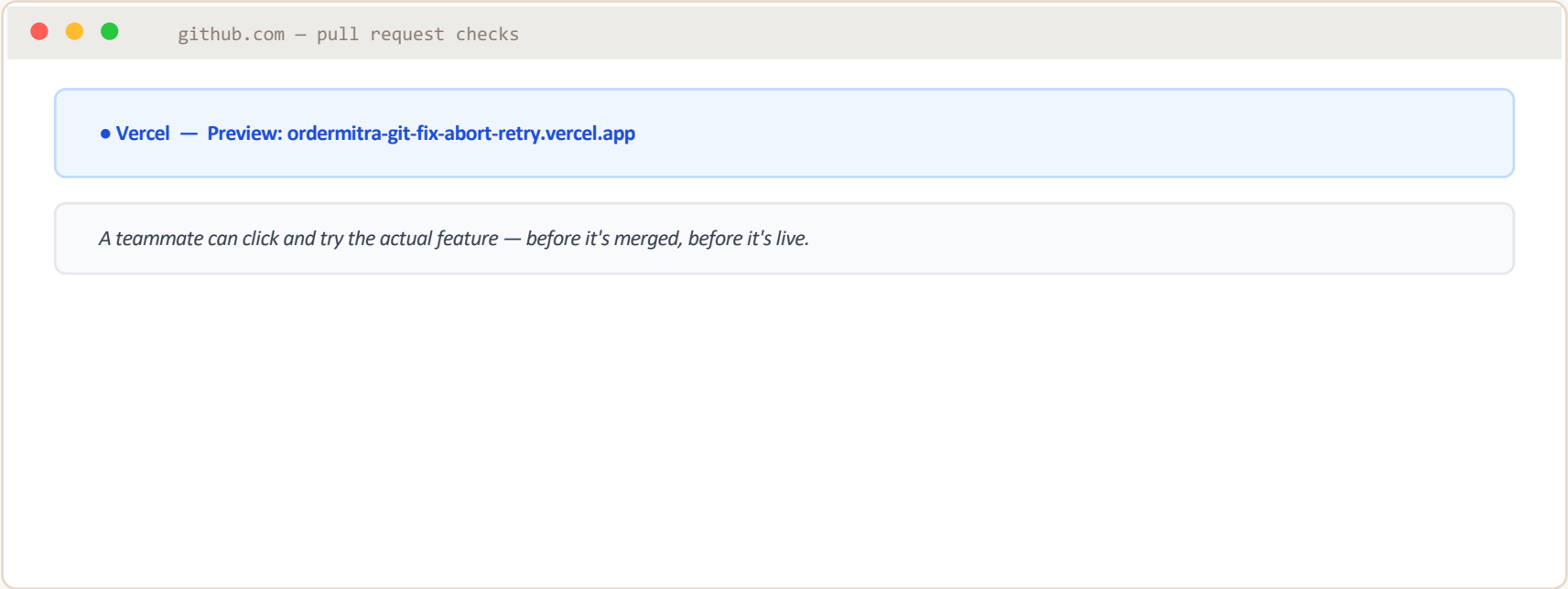
The Automatic Loop

- 1. You push a commit to any branch
- 2. Vercel notices instantly, via its GitHub connection
- 3. It builds the project fresh, from that exact commit
- 4. Push to main → production URL updates
- 5. Push to any other branch or PR → a new preview URL is created instead

No dashboard clicking required ever again — this loop is why Session 4 called Vercel "deployment, built for exactly this framework."

Preview Deployments: Testing Before It's Real

Every pull request gets a live, shareable URL before anything touches production.



This is where a reviewer would actually click Stop and Retry three times fast, before the deploy-day story on the next slide had the chance to happen.

The Complete Setup, End to End

Every exact-steps slide from this chapter, in the order you'd actually do them, once, for a real project.

1. Create the GitHub repo, push your code, add .github/workflows/test.yml
2. Add any real secrets (API keys) under repo Settings → Secrets and variables → Actions
3. Enable branch protection: Settings → Branches → require the Test check
4. Connect the same repo to Vercel: Add New → Project → Import → Deploy
5. Add the same secrets again in Vercel: Settings → Environment Variables
6. Push a change — watch GitHub Actions run, then watch Vercel deploy automatically

Six steps, done once. Every push after this follows the same automatic path, with zero manual work from here on.

A DEPLOY-DAY STORY

The build compiled. Every test was green. It still broke — because the one thing nobody tested was what happens when a real user clicks Stop and Retry, back to back, three times fast.

CI catches what you thought to test for. It has never once caught what nobody thought to write down.

The Last Safety Net: Instant Rollback

CI and CD stop most problems before they ship. This is what happens on the ones they don't.

In Plain Words

- Vercel keeps every past deployment live, not just the current one
- "Promote to Production" on an older deployment reverts everything, instantly
- No new build, no waiting on CI — just pointing the live URL at a version you already know worked

Testing and CI reduce how often you'll need this. They have never once promised zero — rollback is what makes that honest gap survivable.

Where Does AI Actually Fit Into All of This?

A fair question — this chapter has felt more "traditional engineering" than AI-native so far. Here's where AI shows up.

AI Helps With

- Drafting the workflow YAML itself (every code slide this chapter used a prompt)
- Reading a failing CI log and explaining what broke, in plain English
- Suggesting the missing regression test once a drift bug is found
- Drafting Chapters 1-2's tests, which are what CI actually runs

> *AI didn't invent CI/CD — it just makes each individual piece (the YAML, the tests, the debugging) faster to produce. The judgment about what to test and what to gate is still yours.*

Your Thumb Rule: What Belongs in CI for an AI Feature

? Component and mocked-AI tests from Chapter 1?	→ **Always in CI**
? A real, live call to the LLM provider?	→ **Rarely — slow, costs money, non-deterministic**
? Coverage threshold check (e.g. fail under 80%)?	→ **Yes — enforces the habit automatically**
? Deploy to production if any test fails?	→ **Never — that's the entire point of the gate**

This Whole Chapter, Beyond OrderMitra

None of this — Chapter 1, 2, or 3 — is actually about OrderMitra specifically. Worth saying plainly, once, before we close.

The General Practice

- Any React app: Jest + RTL, Arrange/Act/Assert, coverage as a floor
- Any AI feature: mock the model call, test shape not wording, watch for drift
- Any team project: CI on every push, branch protection, gated deploys
- Any production system: rollback as the last safety net, always

> *OrderMitra was the example. This — test locally, let AI help draft, audit what it misses, gate every push through CI, deploy automatically — is the actual, transferable skill.*

Aside: Where Else This Shows Up

CI/CD isn't an AI-native invention — AI features just joined a decades-old discipline.

You've Likely Seen This Already

- Every major open-source project runs CI on every pull request
- Mobile app releases gated behind automated test suites
- Bank and payment systems — nothing ships without passing regression tests
- Session 4 and 5's own repos could use this exact same workflow file

> *OrderMitra just became one more project with a green checkmark gating what reaches real users — the same discipline as everything else worth trusting.*

Chapter 3 Key Terms

Background material worth keeping.

Terms

- Regression — something that used to work, and quietly stopped, without a deliberate change
- CI (Continuous Integration) — tests run automatically on every push
- CD (Continuous Deployment) — shipping automatically once CI passes
- YAML — the plain-text, indentation-based format workflow files are written in
- Branch protection — the setting that makes a green check mandatory, not optional

END OF CHAPTER 3

Shipping It

- A regression is a bug with no matching code change — model updates and drift, not your own mistakes
- CI runs your tests automatically on every push; branch protection makes that mandatory
- YAML defines the workflow; npm ci guarantees identical installs, every run
- CD ships automatically once CI is green; rollback is the safety net for the rest
- AI drafts the YAML and the tests it runs — the judgment about what to gate stays yours

The takeaway: *Nothing in this chapter replaces judgment — CI just guarantees the judgment you already wrote down as tests gets applied every single time, forever.*

🔨 HANDS-ON STARTING

Ship OrderMitra, End to End

~20 min

The whole session, in one pass — draft tests, find the gap, fix it, push, watch it deploy.

HANDS-ON COMPLETE

OrderMitra Is Tested, Verified, and Live!

Every chapter from today, run for real, on your own repo. Back to the lecture for the recap.